

NTP 2026 · SpaceSec Shield

SpaceSec Shield

Phase 5 — Monitoring Dashboard

Implementation & Validation Report

Field	Value
Task ID	SSS-P5
Phase	5 of 5 — Real-Time Monitoring & Control Interface
Priority	High (the visible proof — first thing judges see)
Dependencies	SSS-P3 (security events) · SSS-P4 (attacks to visualize)
Status	COMPLETED
Prepared for	SpaceSec Shield Project Evaluation
Prepared by	SpaceSec Shield Team
Date	June 2026

Technology Stack

Python · Flask · Chart.js · HTTP/JSON · C++ (raw-socket client) · ChaCha20-Poly1305 AEAD link

Contents

1. Executive Summary

Phase 5 delivers the human-machine interface for SpaceSec Shield: a centralized, real-time web dashboard that makes the security layer visible. The Satellite Node — which already decides accept or reject for every command packet using the Phase 3 logic (ChaCha20-Poly1305 AEAD, anti-replay, timestamp freshness) — now reports each decision to the dashboard. An operator can watch legitimate commands raise the accepted line, and watch a Phase 4 attack produce an instant, colour-coded CRITICAL alert the moment it is rejected.

The dashboard back end is a lightweight Flask server that exposes a small REST API; the front end is a single Chart.js page that polls the server once per second. To connect the C++ Satellite Node to the Python dashboard without adding heavyweight dependencies, a dependency-free HTTP client was written that posts JSON over a raw TCP socket. The client is best-effort with a short non-blocking timeout, so a slow or offline dashboard can never stall the Satellite's packet loop.

The system was deployed and validated in the official three-VM space-net lab on VMware Workstation. In the live demonstration the dashboard correctly visualized two accepted commands and three rejected attacks (duplicate-nonce replay, tampered payload, and expired timestamp), classified all three as CRITICAL, and flipped the link status to DEGRADED — exactly the intended outcome.

2. Introduction and Objective

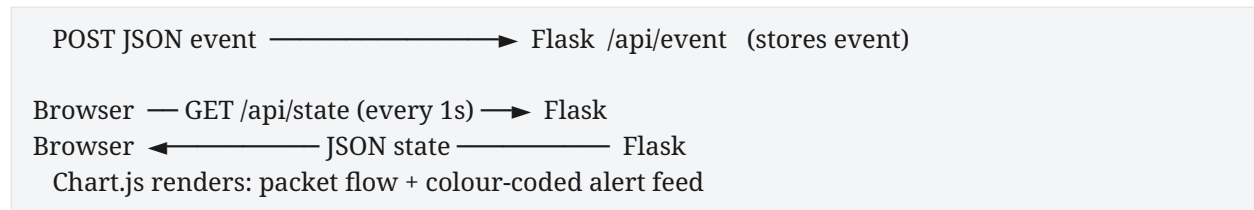
By the end of Phase 4, SpaceSec Shield was a complete, validated secure-communication stack: a stable two-way link (Phases 1–2), authenticated encryption (Phase 3), and a documented penetration-test campaign proving the channel resists eavesdropping, injection, replay, and tampering (Phase 4). What was still missing was visibility. The security mechanisms worked, but their results lived only in terminal logs that scroll past and are hard to present.

The objective of Phase 5 is to build a real-time dashboard that surfaces three things at a glance: live communication status, a security-event stream, and integrity-verification results. The dashboard observes and reports — it does not enforce. Enforcement already happens on the Satellite Node (Phase 3); the dashboard's job is to make those decisions legible to a human operator and to a panel of judges.

3. Architecture and Design Decisions

The architecture keeps a clean, one-directional reporting path: the Satellite Node pushes events to the dashboard, and the browser pulls aggregated state from the dashboard. No security logic is duplicated.

Satellite Node (Phase 3 logic)
on every packet: decide ACCEPT / REJECT



The locked design decisions are summarized below. The final row records the one new engineering decision introduced in this phase to bridge the C++ security layer and the Python dashboard.

Concern	Choice	Rationale
Back end	Python + Flask	Lightweight web server with simple REST endpoints.
Charts	Chart.js (in-browser)	Smooth live updates without re-rendering image files; far better than Matplotlib for real time.
Data ingestion	HTTP POST: Satellite → Flask	The Satellite already decides per packet (Phase 3); it posts each event directly. Clean, direct link between the security layer and the dashboard.
Live refresh	Polling (fetch every 1 s)	Simple, robust, and sufficient for a live demo (SSE/WebSocket is an optional upgrade).
Satellite → dashboard transport	Dependency-free C++ client (raw TCP HTTP POST), best-effort, 300 ms non-blocking timeout	Avoids adding libcurl to the satellite build; a slow/offline dashboard never stalls the packet loop.

Terminology alignment (Phases 3 & 4): integrity failures are reported as “*AEAD tag verification failed*” (reason TAG_FAIL) — never “hash mismatch” — keeping the dashboard consistent with the ChaCha20-Poly1305 design.

4. Event Schema (Satellite ↔ Dashboard Contract)

Every decision the Satellite makes is reported as one JSON object. This is the contract between the security layer and the dashboard.

```

{
  "timestamp": 1716300000000,
  "type": "ACCEPTED | REJECTED",
  "reason": "OK | TAG_FAIL | REPLAY_TS | DUP_NONCE | DECRYPT_FAIL | BAD_FORMAT",
  "session_id": "07EA",
  "latency_ms": 480,
  "severity": "LOW | MEDIUM | CRITICAL"
}
  
```

Field	Meaning
-------	---------

type	Whether the packet was accepted (and executed) or rejected.
reason	Why: AEAD tag fail, replay by timestamp, duplicate nonce, malformed packet, or OK.
session_id	Session identifier; the uint32 session id is formatted as a 4-digit hex string (e.g. 2026 → 07EA).
latency_ms	now – packet.timestamp_ms; a rough one-way staleness of the packet.
severity	Drives the alert colour/level on the dashboard (set on the Satellite side).

The Satellite sets the severity; the dashboard maps it to a colour. The reason-to-severity mapping is:

Reason	Severity	Trigger
OK	LOW	Accepted, authenticated command.
BAD_FORMAT	MEDIUM	Malformed / truncated packet (e.g. a raw netcat injection).
REPLAY_TS	CRITICAL	Timestamp outside the freshness window.
DUP_NONCE	CRITICAL	Nonce already seen this session (replay).
TAG_FAIL	CRITICAL	Poly1305 AEAD tag failed (tampered or forged).

5. System Components and Implementation

5.1 Satellite-Side Hook (modified `secure_satellite_node.cpp`)

The only change to the Phase 3 security code is a reporting hook. Inside the packet-processing routine, a single call is added at each decision point. The decision logic itself is untouched — the Satellite still rejects bad packets exactly as before; it now also reports what it did.

```
// freshness check failed (stale / replayed-by-time)
post_security_event("REJECTED", "REPLAY_TS", session_id, latency_ms, "CRITICAL");
// nonce already seen this session
post_security_event("REJECTED", "DUP_NONCE", session_id, latency_ms, "CRITICAL");
// AEAD tag verification failed (tampered / forged)
post_security_event("REJECTED", "TAG_FAIL", session_id, latency_ms, "CRITICAL");
// authenticated command accepted and executed
post_security_event("ACCEPTED", "OK", session_id, latency_ms, "LOW");
```

A single initializer in `main()` — `dashboard_init("192.168.10.10", 5000)` — sets where events are sent.

5.2 Dashboard Client (new: `dashboard_client.cpp` / `.hpp`)

The Satellite is C++ and the dashboard is Python, so the Satellite needs to issue an HTTP POST. Rather than link libcurl (an extra dependency on the constrained node, and an extra build flag), a small client was written that builds a minimal HTTP/1.1 request by hand and sends it over a raw TCP socket. Two engineering properties make it safe for a flight-like loop:

- Non-blocking connect with a `select()` timeout, plus send/receive timeouts — all capped at 300 ms. If the dashboard is unreachable, the call returns quickly instead of waiting on the default TCP connect timeout.
- Best-effort semantics: any failure is swallowed silently. Monitoring must never be able to crash or stall the security node. This was verified by running the Satellite with the dashboard offline (see TC-07).

This keeps the Satellite build a single line with no new libraries:

```
g++ -std=c++17 secure_satellite_node.cpp handshake.cpp crypto.cpp \
    packet.cpp anti_replay.cpp dashboard_client.cpp \
    -o secure_satellite -lssl -lcrypto
```

5.3 Flask Back End (`dashboard.py`)

The back end keeps everything in memory (no database is needed for the demo) and exposes four routes:

Route	Method	Purpose
<code>/api/event</code>	POST	Satellite posts one packet-decision event; the server stores it and updates counters.
<code>/api/state</code>	GET	Browser polls this every second; returns link status, cumulative accepted/rejected, average latency, and recent events.
<code>/api/reset</code>	POST	Clears all state so each live demo run starts from zero.
<code>/</code>	GET	Serves the dashboard page.

Cumulative counters keep the chart line rising for the whole demo, average latency is computed only over accepted packets, and link status is derived from recent activity: **CONNECTED** when traffic is mostly accepted, **DEGRADED** when recent rejections dominate, and **DISCONNECTED** before any event arrives.

5.4 Front End (`templates/index.html`)

A single self-contained page loads Chart.js from a CDN and polls `/api/state` once per second. It shows a colour-coded status banner, three stat cards (Accepted, Rejected, Average latency), a live accepted-vs-rejected line chart, and a security-alert feed. New **CRITICAL** events flash the status banner red. A Reset button clears state for clean demo runs.

5.5 Alert Classification

Severity is decided on the Satellite and shown on the dashboard. The three Phase 4 attack signatures (TAG_FAIL, REPLAY_TS, DUP_NONCE) are CRITICAL and render red; a malformed injection is MEDIUM (amber); accepted traffic is LOW (blue). The dashboard reports and the Satellite enforces — the packet is already rejected by the time the alert is drawn.

6. Deployment and Run Procedure

The dashboard was deployed in the same isolated three-VM lab used in earlier phases.

VM	Role in Phase 5	IP	Phase 5 components
Ground Station	Mission control / secure sender + dashboard host	192.168.10.10	dashboard.py, templates/index.html, Flask :5000
Satellite Node	Secure receiver; posts a decision event per packet	192.168.10.20	secure_satellite (+ dashboard_client.cpp), interface ens33
Kali	Attacker (Phase 4 attacks visualized live)	192.168.10.30	tcpreplay / netcat / hping3

Run order (the receiver must be up before the sender, as in Phases 1–2):

1. On the Ground Station: pip3 install flask, then python3 dashboard.py (serves on 0.0.0.0:5000). Open `http://localhost:5000` — status starts DISCONNECTED.
2. On the Satellite Node: build with dashboard_client.cpp added, then run `./secure_satellite`. It listens on UDP 9000 and re-handshakes whenever the Ground Station restarts.
3. On the Ground Station: run `./secure_ground` (unchanged from Phase 3). The handshake completes and the built-in command/attack sequence begins.

A quick reachability check from the Satellite confirms the link before a demo:

```
curl -X POST http://192.168.10.10:5000/api/event \
-H "Content-Type: application/json" \
-d '{"type":"REJECTED","reason":"TAG_FAIL","severity":"CRITICAL","session_id":"07EA"}
```

7. Testing and Results

The live demonstration tied Phases 3–5 together. After the secure handshake, the Ground Station sent two legitimate commands and three attacks. The Satellite enforced the Phase 3 policy and reported every decision; the dashboard visualized them in real time. The Flask access log confirmed POST /api/event requests arriving from 192.168.10.20 (the Satellite), proving end-to-end reporting.

Test	Action	Observed result	Status
TC-01	Reachability: curl POST from Satellite to dashboard	200 OK; alert appears on the page	PASS
TC-02	Valid command (CMD:CAPTURE_IMAGE)	ACCEPTED/OK event; green line rises; status CONNECTED	PASS
TC-03	Replay (same packet resent)	REJECTED/DUP_NONCE; red CRITICAL alert	PASS
TC-04	Tampered payload (flipped byte)	REJECTED/TAG_FAIL; red CRITICAL alert	PASS
TC-05	Expired timestamp packet	REJECTED/REPLAY_TS; red CRITICAL alert	PASS
TC-06	Second valid command (CMD:STATUS_CHECK)	ACCEPTED/OK; Accepted=2, Rejected=3; status DEGRADED	PASS
TC-07	Dashboard offline during operation	Satellite keeps running; posts fail fast (<300 ms); no crash	PASS
TC-08	Heavy netem (800 ms / 50% loss) on ens33	Monitoring channel degraded; documented honestly (Section 8)	NOTED

Aggregate dashboard state at the end of the run: Accepted = 2, Rejected = 3, average latency ≈ 400 ms, link status = DEGRADED, with three CRITICAL alerts in the feed (DUP_NONCE, TAG_FAIL, REPLAY_TS). The chart showed the green accepted line and red rejected line rising and then holding once the sender finished.

7.1 Evidence

[Paste screenshot: dashboard at end of run — Accepted 2 / Rejected 3, status DEGRADED, three red CRITICAL alerts]

Figure 1 — Live dashboard: two accepted commands, three CRITICAL attack alerts, link status DEGRADED.

[Paste screenshot: Satellite terminal — [ACCEPT]/[REJECT] decisions and Flask log showing POST /api/event from 192.168.10.20]

Figure 2 — Satellite enforcement log and the Flask access log confirming events arriving from the Satellite (192.168.10.20).

[Paste screenshot: Ground Station terminal — handshake, valid commands ACKed, attacks timing out (no ACK)]

Figure 3 — Ground Station: valid commands acknowledged; attack packets receive no ACK (rejected by the Satellite).

8. Interaction with the Space-Environment Simulation (tc/netem)

The Phase 2 space-link simulation (Linux tc/netem on ens33) and the Phase 5 live dashboard are presented as two separate moments, not run together, for a deliberate reason. The Satellite reports to the dashboard over the same ens33 interface that carries the command link. When heavy impairment is applied (for example delay 800 ms / loss 50%), the monitoring POSTs are delayed and dropped just like everything else; because the client uses a 300 ms best-effort timeout, reports that exceed it are abandoned (so the Satellite is never stalled) and the dashboard stops updating.

This is not a defect — it is an honest demonstration of the Phase 4 finding that availability is outside the scope of a cryptographic layer. Two presentation options follow from it:

- Separate the two parts of the demo: show the netem stability proof (Phase 2), remove the qdisc, then show the live alerts (Phase 5). This is the cleanest narrative for the submission.
- Use a light impairment that stays under the timeout (e.g. delay 150 ms $\pm$ 30 ms, loss 5%) while the dashboard is live; most reports still arrive, and the elevated average-latency figure becomes useful visual evidence that the link is degraded yet the system keeps functioning.

In all cases the security guarantees are unaffected: a delayed packet that still arrives within the 30-second freshness window is processed normally, and confidentiality and integrity continue to hold regardless of impairment.

9. Challenges

- Bridging C++ and Python without new dependencies. Solved with a hand-built HTTP POST over a raw socket instead of libcurl, keeping the Satellite build unchanged except for one extra source file.
- Preventing monitoring from harming the security node. Solved with a 300 ms non-blocking, best-effort client; verified by running with the dashboard offline.
- Re-handshake handling. When the Ground Station restarts between demo runs, it sends a fresh public key; the Satellite detects the 32-byte handshake, re-derives a new session

key, and resets its anti-replay state so the new session's nonces are not mistaken for replays.

- Demo hygiene. The monitoring channel shares the impaired interface; this was turned into a documented finding (Section 8) and a Reset button was added for clean runs.

10. Limitations and Future Work

- Availability under heavy jamming/DoS remains outside the crypto layer; the monitoring channel is impacted along with the command link (consistent with Phase 4).
- State is in-memory only. A persistent store (and a separate monitor VM) would be needed for long missions and post-event forensics.
- Polling at 1 s is simple and robust for a demo; Server-Sent Events or WebSockets would lower latency for production.
- The dashboard transport is plain HTTP on the isolated lab network. On an operational network the Satellite → dashboard link should itself be authenticated/encrypted.
- MEDIUM-severity escalation (repeated rejects from one session) is defined but minimally exercised; richer correlation could be added on the dashboard side.

11. Conclusion

Phase 5 completes SpaceSec Shield. The system now spans the full cycle: a stable two-way link, realistic space-link conditions, authenticated encryption, validation against real attacks, and live observation. The dashboard turns the Satellite's security decisions into something an operator — or a panel of judges — can see in real time: legitimate commands raise the accepted line, and each Phase 4 attack produces an immediate, correctly classified CRITICAL alert as the link drops to DEGRADED.

The implementation respected the project's guiding constraint throughout. The reporting path adds no heavyweight dependency to the resource-constrained node and cannot stall or crash it. The result is a complete, demonstrable secure-communication framework for a CubeSat-class platform, validated end-to-end in the official three-VM environment.

12. Deliverables and Definition of Done

Item	Status
dashboard.py (Flask: /api/event, /api/state, /api/reset, /)	Delivered
templates/index.html (Chart.js live UI)	Delivered
dashboard_client.hpp / .cpp (dependency-free POST client)	Delivered

Item	Status
Satellite-side hook posting an event per packet decision	Delivered
Documented event JSON schema	Delivered
Real-time updates (1 s polling)	Verified
Attack → immediate, correctly classified CRITICAL alert	Verified
System stable under continuous monitoring / dashboard offline	Verified
Demo recording: attack → instant CRITICAL alert	To record during your run